
Software Requirements Specification

for

Rise, Tarnished

Version 1.0 approved

Prepared by Mustakim Ahmed Hasan,

Md Rafiul Haque,

Md Mafiul Haque

ECE Dept, North South University

21 August 2026

Table of Contents

Table of Contents	ii
Revision History	ii
1. Introduction	1
1.1 Purpose	1
1.2 Document Conventions	1
1.3 Intended Audience and Reading Suggestions	1
1.4 Product Scope	1
1.5 References	1
2. Overall Description	2
2.1 Product Perspective	2
2.2 Product Functions	2
2.3 User Classes and Characteristics	2
2.4 Operating Environment	2
2.5 Design and Implementation Constraints	2
2.6 User Documentation	2
2.7 Assumptions and Dependencies	3
3. External Interface Requirements	3
3.1 User Interfaces	3
3.2 Hardware Interfaces	3
3.3 Software Interfaces	3
3.4 Communications Interfaces	3
4. System Features	4
4.1 System Feature 1	4
4.2 System Feature 2 (and so on)	4
5. Other Nonfunctional Requirements	4
5.1 Performance Requirements	4
5.2 Safety Requirements	5
5.3 Security Requirements	5
5.4 Software Quality Attributes	5
5.5 Business Rules	5
6. Other Requirements	5
Appendix A: Glossary	5
Appendix B: Analysis Models	5
Appendix C: To Be Determined List	6

Revision History

Name	Date	Reason For Changes	Version
Mustakim Ahmed Hasan	21 August 2026	Initial SRS prepared from the project proposal and analysis models	1.0

1. Introduction

1.1 Purpose

This Software Requirements Specification defines Rise, Tarnished version 1.0. Rise, Tarnished is a two dimensional single player web based boss fighting game. The player progresses through a fixed sequence of four boss encounters, earns runes by defeating bosses, and spends those runes to improve character attributes. This SRS covers authentication, progression, character development, combat, adaptive boss behavior, per attempt variation, combat resolution, persistent player data, and post death analysis.

1.2 Document Conventions

The document follows the structure of the supplied IEEE style SRS template. Functional requirements use unique REQ identifiers and are written as verifiable statements. High, Medium, and Low indicate feature priority. The term player refers to the person using the game. The term system refers to the web application and its supporting services. Requirements are written so that each stated capability can be verified through testing or inspection.

1.3 Intended Audience and Reading Suggestions

This document is intended for the student development team, project supervisors, testers, and reviewers of Rise, Tarnished. Developers should read Sections 1 and 2 before reviewing the detailed requirements. Testers should focus on Section 4 and the measurable requirements in Section 5. Reviewers can use Sections 1 and 2 together with Appendix B to understand the overall design. Appendix A provides definitions for the main game and implementation terms.

1.4 Product Scope

Rise, Tarnished provides a focused boss fighting experience in which progress depends on defeating the current boss. The player can develop a build using vitality, health, dexterity, and intelligence. Bosses respond to recent player behavior and can vary their attack sequencing between attempts. The goal is to make repeated encounters challenging and learnable without making them completely predictable. The product also provides persistent progression and a short explanation of the main cause of death after an unsuccessful attempt.

1.5 References

Project proposal for Rise, Tarnished, CSE327.9, North South University, version supplied for this project. The proposal defines the game concept, progression, major features, and technology stack. IEEE Software Requirements Specification template based on the provided Karl E. Wiegers template, version 1.0 approved. The supplied activity diagram, class diagram, communication diagram, combat sequence diagram, system sequence diagram, and use case diagram are supporting analysis models for this SRS.

2. Overall Description

2.1 Product Perspective

Rise, Tarnished is a new self contained web based game. The game shell uses Next.js and the combat experience uses Phaser.js. Supabase provides authentication and PostgreSQL data storage. Google OAuth is the sole sign in method. Real time boss decisions are handled inside the game through a finite state machine because frame level reactions cannot depend on external model latency. The OpenAI API is used for asynchronous post death analysis and for move sequencing weights. Vercel hosts the application while Supabase provides managed database and authentication services. The main runtime flow begins with authentication, loads persistent player data, enters the current boss encounter, processes combat input, resolves the result, updates progression when appropriate, and stores attempted data.

2.2 Product Functions

The major product functions are

- *Sign in with Google and establish a player session*
- *Load and resume player progression*
- *Display the character sheet and current build*
- *Spend available runes on character attributes*
- *Enter the next unlocked boss encounter*
- *Accept supported movement and combat input*
- *Resolve player and boss actions using the current combat state*
- *Adapt boss behavior using recent player combat patterns*
- *Vary boss move selection between attempts within fairness constraints*
- *Award runes and unlock the next region after a successful boss defeat*
- *Store relevant player statistics and combat attempt data*
- *Provide a post death explanation based on the recorded attempt*
- *Display the final outcome of each encounter*

2.3 User Classes and Characteristics

The primary user class is the Player. The player is expected to understand basic web navigation and common action game controls. No specialist technical knowledge is required. The player can authenticate, manage character attributes, fight bosses, and review progression and combat outcomes.

The development and testing team is a secondary user class. This group needs to verify authentication, progression, combat resolution, adaptive behavior, persistence, and failure handling. These activities are development and testing responsibilities rather than normal player functions.

2.4 Operating Environment

The product operates as a web application in a modern desktop web browser. The game shell uses Next.js and the combat engine uses Phaser.js. Authentication uses Supabase Auth with Google

OAuth. Persistent data is stored in PostgreSQL through Supabase. The application is hosted on Vercel. The OpenAI API is accessed for asynchronous AI functions. The system should operate on common desktop hardware capable of running a modern browser and rendering a two dimensional Phaser.js game.

2.5 Design and Implementation Constraints

The game must remain web based and use the project technology stack defined in the proposal. Google OAuth is the only sign in method. Real time combat decisions must be handled locally within the game logic and finite state machine rather than waiting for an external language model. Persistent player data must be stored through Supabase and PostgreSQL. The combat loop must support frame level input and state updates. External AI calls are limited to asynchronous functions such as post death analysis and move sequencing support. The game follows linear progression and does not provide free boss selection.

2.6 User Documentation

The delivered product should provide concise in game guidance for the main controls, character upgrades, progression, and combat outcome. The character sheet and combat interface should provide enough contextual information for a player to understand available actions and current progression without requiring technical documentation. A short project level user guide may be provided separately if required by the course deliverables.

2.7 Assumptions and Dependencies

The system assumes that the player has a stable internet connection for authentication, persistent data access, hosting, and asynchronous API services. The system depends on Google OAuth, Supabase Auth, Supabase PostgreSQL, Vercel hosting, and the OpenAI API. The system assumes that the target browser supports the required Next.js and Phaser.js functionality. The adaptive boss system depends on the combat log and recent player behavior being available during an encounter. Some boss balancing values and behavior tuning may change during implementation while preserving the functional requirements in this document.

3. External Interface Requirements

3.1 User Interfaces

The user interface consists of the sign in screen, main game view, character sheet, progression information, combat HUD, and combat outcome information. The sign in screen provides a clear Google sign in action. The character sheet displays available runes and character attributes and allows valid stat upgrades. The combat screen displays the player and boss state needed to make informed decisions. Combat controls respond consistently to supported keyboard input. Invalid or unavailable actions do not corrupt player state. The interface communicates successful upgrades, insufficient runes, boss defeat, player death, and progression changes clearly. The supplied use case and sequence models describe important interactions and are reserved for Appendix B.

3.2 Hardware Interfaces

The product does not require a dedicated external hardware interface. A standard desktop computer with a keyboard and display is sufficient for the intended game experience. Keyboard input is used to control supported combat actions. The browser and Phaser.js provide the software level connection between user input and game logic.

3.3 Software Interfaces

Next.js provides the web application and game shell. Phaser.js provides the combat scene, rendering, animation, combat timing, and hitbox and hurtbox handling. Supabase Auth manages authentication and Google OAuth sessions. Supabase PostgreSQL stores player statistics, progression, builds, and attempt data. The OpenAI API supports asynchronous post death analysis and move sequencing weights. Vercel provides application hosting. Authenticated player identity and persistent player state are shared between the game shell and Supabase services. Combat state is maintained in the game runtime and relevant results are persisted after resolution.

3.4 Communications Interfaces

The application communicates with external services over standard web protocols. Authentication and data operations use HTTPS. The browser communicates with the hosted application through normal web requests. Supabase provides authenticated access to database and authentication services. The OpenAI API is accessed over HTTPS for asynchronous AI operations. Requests should fail safely when a remote service is unavailable. A temporary external service failure must not create an invalid character state or grant duplicate progression.

4. System Features

The functional requirements are organized around the major services that define the player experience. These services are authentication and progression, character development, boss combat, adaptive behavior and attempt variation, and combat resolution with post death analysis.

4.1 Authentication and Progression

4.1.1 Description and Priority

This feature manages player sign in, session establishment, progression loading, and access to the game. It is the entry point for a new or returning player.

Priority High

4.1.2 Stimulus/Response Sequences

- 1. The player selects Sign in with Google.*
- 2. The system redirects the player to Google for authentication and consent.*
- 3. Google returns the authentication result to the application callback.*
- 4. Supabase establishes the authenticated session.*
- 5. If the account is new, the system creates the required player records.*
- 6. The system redirects the authenticated player to the game.*

7. The game loads the stored character, build, and progression data.
8. If the player is not authenticated, protected game data is not loaded.

4.1.3 Functional Requirements

The system shall provide Google OAuth as the sole player sign in method.
The system shall establish a Supabase authenticated session after successful Google authentication.
The system shall create the required player records for a first time authenticated account.
The system shall load the authenticated player saved progression when the game is opened.
The system shall prevent unauthenticated access to protected player data.
The system shall preserve stored progression across sessions.
The system shall handle authentication failure without creating partial player records.
The system shall prevent duplicate initialization of an existing player account.

Each requirement is uniquely identified below.

REQ 1 The system shall provide Google OAuth as the sole player sign in method.

REQ 2 The system shall load the authenticated player saved progression before protected gameplay begins.

4.2 Character Development and Rune Spending

4.2.1 Description and Priority

The player can use earned runes to improve the character. The feature supports vitality, health, dexterity, and intelligence as the primary build attributes.

Priority High

4.2.2 Stimulus/Response Sequences

1. The player opens the character sheet.
2. The system displays current attributes, available runes, and upgrade costs.
3. The player selects an attribute upgrade.
4. The system checks the available rune balance.
5. If enough runes are available, the system applies the upgrade and persists with the new values.
6. If the player lacks enough runes, the system rejects the request and leaves the character unchanged.
7. The updated character sheet is rendered after a successful upgrade.

4.2.3 Functional Requirements

- REQ 3 The system shall display the player current rune balance and supported character attributes.
- REQ 4 The system shall allow the player to spend available runes on valid attribute upgrades.
- REQ 5 The system shall reject an upgrade when the rune balance is below the required cost.
- REQ 6 The system shall persist successful attribute upgrades.
- REQ 7 The system shall prevent an upgrade from reducing the rune balance below zero.
- REQ 8 The system shall update the character sheet after a successful stat upgrade.

4.3 Boss Combat

4.3.1 Description and Priority

This feature provides the main gameplay loop. The player fights the currently unlocked boss using supported combat actions while the game continuously updates player and boss state.

Priority High

4.3.2 Stimulus/Response Sequences

1. The player enters the current boss encounter.
2. The game initializes the player and boss combat state.
3. The player provides keyboard input during the combat loop.
4. The combat scene converts input into combat actions.
5. Player actions and boss actions are resolved using the current state.
6. The boss tactic system evaluates recent combat signals.
7. The game updates health, stamina, posture, cooldowns, and other applicable state.
8. The game determines whether the encounter continues or has reached an outcome.
9. The final state is rendered to the player.

4.3.3 Functional Requirements

REQ 9 The system shall allow the player to control supported combat actions through the game input system.

REQ 10 The system shall update combat state during the active combat loop.

REQ 11 The system shall resolve player attacks against the boss using the current player and boss state.

REQ 12 The system shall resolve boss attacks against the player using the current boss state.

REQ 13 The system shall prevent invalid combat state transitions such as applying damage to an already defeated combatant.

REQ 14 The system shall determine when the player or boss has reached the conditions for an encounter outcome.

REQ 15 The system shall render current combat state and outcome information to the player.

4.4 Adaptive Boss Behavior and Attempt Variation

4.4.1 Description and Priority

This feature changes boss decision making according to recent player behavior. The boss uses an in engine finite state machine for real time decisions and can vary move sequencing between attempts within defined fairness constraints.

Priority High

4.4.2 Stimulus/Response Sequences

1. The combat loop records relevant player actions and combat signals.
2. The tactic system updates behavior signals during the encounter.

3. The boss evaluates the current tactic state and available actions.
4. The action selection logic chooses a valid move or approach.
5. The boss executes the selected behavior.
6. On later attempts, move sequencing can be reweighted using recorded attempt information.
7. The system keeps selection within the available boss move set and fairness constraints.

4.4.3 Functional Requirements

- REQ 16 The system shall track relevant recent player combat behavior during an active encounter.
- REQ 17 The system shall use tracked behavior to influence boss tactic selection.
- REQ 18 The system shall make real time boss decisions through the in engine finite state machine.
- REQ 19 The system shall select boss actions only from actions available to the current boss state.
- REQ 20 The system shall support variation in boss move sequencing between attempts.
- REQ 21 The system shall keep move variation within the configured boss behavior and fairness constraints.
- REQ 22 The system shall skip action selection when the boss state requires a non selection approach such as recovery.

4.5 Combat Resolution and Post Death Analysis

4.5.1 Description and Priority

This feature finalizes each attempt, persists the result, awards progression when the boss is defeated, and provides a short explanation after player death.

Priority High

4.5.2 Stimulus/Response Sequences

1. The combat loop reaches a terminal player or boss state.
2. The system determines the fighting outcome.
3. The system persists the relevant attempt result.
4. If the boss is defeated, the system grants runes and unlocks the next region.
5. If the player dies, the system sends the recorded attempt information for asynchronous post death analysis.
6. The system displays the combat outcome.
7. When the analysis is available, the system displays a concise explanation of the main cause of death.

4.5.3 Functional Requirements

- REQ 23 The system shall determine the final outcome of each combat attempt.
- REQ 24 The system shall persist the relevant attempt result and progression data.
- REQ 25 The system shall grant runes when the player defeats the current boss.
- REQ 26 The system shall unlock the next region only after the required boss is defeated.
- REQ 27 The system shall prevent progression from being granted more than once for the same successful resolution.
- REQ 28 The system shall submit recorded combat information for asynchronous post death

analysis after player death.

REQ 29 The system shall present a short explanation of the main cause of death when analysis is available.

REQ 30 The system shall preserve the combat result if the asynchronous analysis service is unavailable.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

REQ 31 The combat loop shall process supported player input without waiting for external AI responses.

REQ 32 Real time boss decisions shall be handled by the in engine finite state machine.

REQ 33 The system shall avoid duplicate progression when the same combat resolution is processed more than once.

REQ 34 The character sheet shall reflect a successful stat update after the corresponding persistent operation completes.

REQ 35 Asynchronous post death analysis shall not block presentation of the resolved combat outcome.

5.2 Safety Requirements

The product is a game and does not control physical equipment. The main safety concern is safe handling of user data and reliable failure behavior. A network or service failure shall not leave a player with a partially applied stat upgrade or duplicated progression. The game shall not present a successful save when the corresponding persistent operation has failed.

5.3 Security Requirements

REQ 36 The system shall require an authenticated session before accessing protected player data.

REQ 37 Player records shall be associated with the authenticated user identity.

REQ 38 The system shall prevent one authenticated player from modifying another player protected progression through normal application operations.

REQ 39 Authentication and external service communication shall use HTTPS.

REQ 40 The system shall not expose private authentication credentials or service secrets to the browser.

REQ 41 Database operations that modify player progression shall enforce the authenticated user ownership of the affected record.

5.4 Software Quality Attributes

Usability: Main player actions should be understandable without technical knowledge.

Reliability: Persistent progression should remain consistent across sessions and failed remote operations should fail safely.

Maintainability: Combat logic, boss tactics, authentication, persistence, and asynchronous AI services should remain separated so that one area can be changed without unnecessary changes to unrelated areas.

Testability: Functional requirements should be testable through controlled game states, player inputs, and inspectable persistent results.

Portability: The game should remain usable on common desktop browsers that support the required web technologies.

Responsiveness: Real time combat should not depend on the response time of the OpenAI API.

Correctness: Rune spending, stat changes, boss defeat, death, and progression should produce consistent and verifiable state transitions.

5.5 Business Rules

The player must defeat the current boss before the next region becomes available.

The player cannot freely select a later boss.

Runes are earned through successful boss encounters and are used for character development.

Character upgrades must respect the current rune balance.

The boss may adapt to recent player behavior, but its actions remain constrained by the configured boss behavior.

Each successful boss defeat grants the associated progression only once.

Post death analysis is supplementary to the combat result and does not determine whether the player actually died.

6. Other Requirements

The system shall store player statistics, character build data, region progress, and relevant combat attempt information in the project database. Stored data shall be sufficient to restore player progression and support post death analysis.

The application should allow additional bosses, moves, and balance values to be added without redesigning the complete authentication and persistence system.

The game is intended for English language presentation unless project requirements are expanded later.

Appendix A: Glossary

Adaptive Boss AI: Boss behavior that changes according to recent player actions and combat signals.

Attempt: A single player runs against a boss that ends in player death or boss defeat.

Boss: The current enemy that blocks linear progression.

Build: The combination of character attribute values that affects how the player performs in combat.

Cooldown: A period during which an action cannot be used again.

Fighting Outcome: The final result of a combat attempt.

Finite State Machine: The in engine state based logic used to select and execute boss behavior during real time combat.

Google OAuth: The authentication mechanism used by the game to allow players to sign in with Google.

Post Death Analysis: An asynchronous explanation generated from the recorded combat attempt after the player dies.

Region: A progression stage associated with a boss encounter.

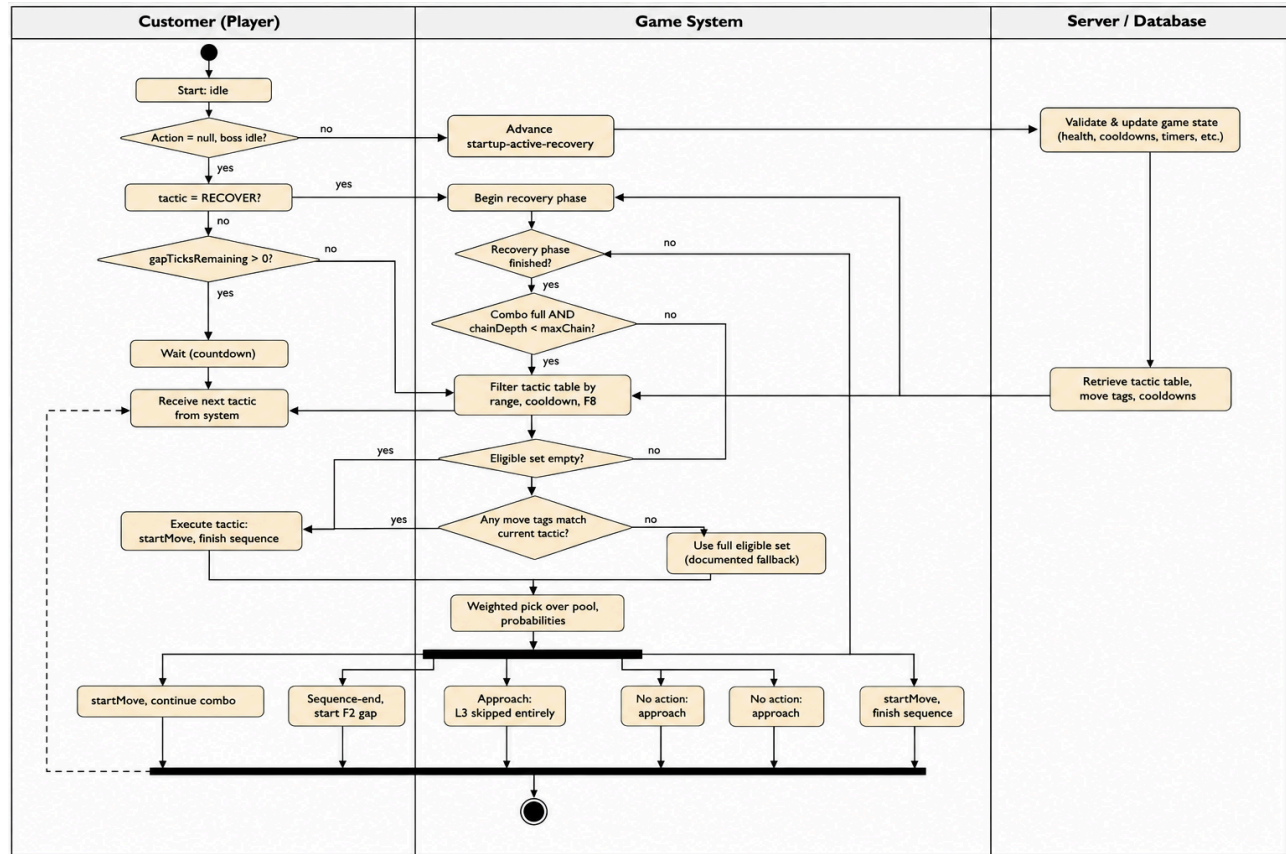
Runes: The in game resource earned from successful boss encounters and spent on character upgrades.

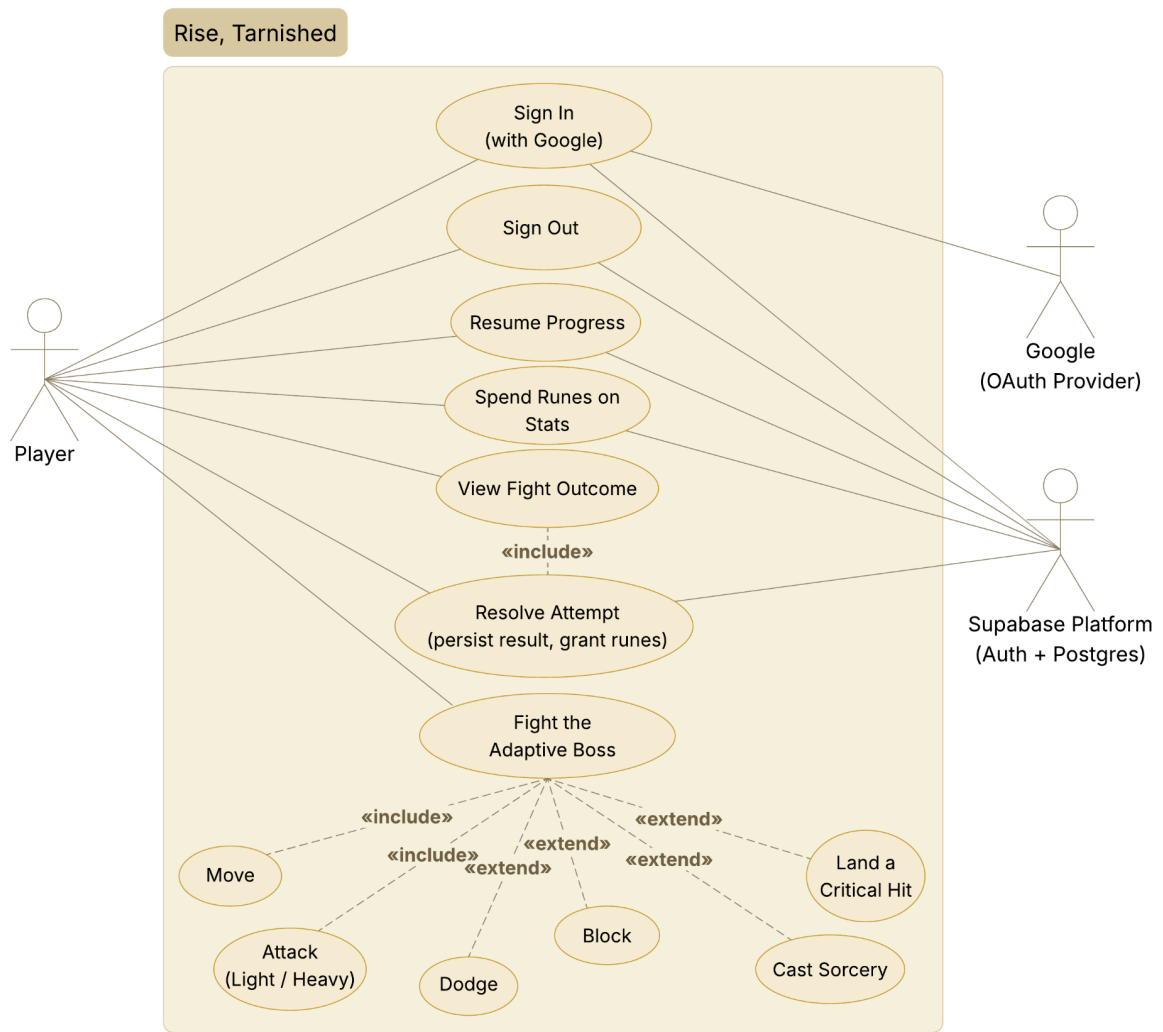
Tactic: The current behavioral strategy used by the boss.

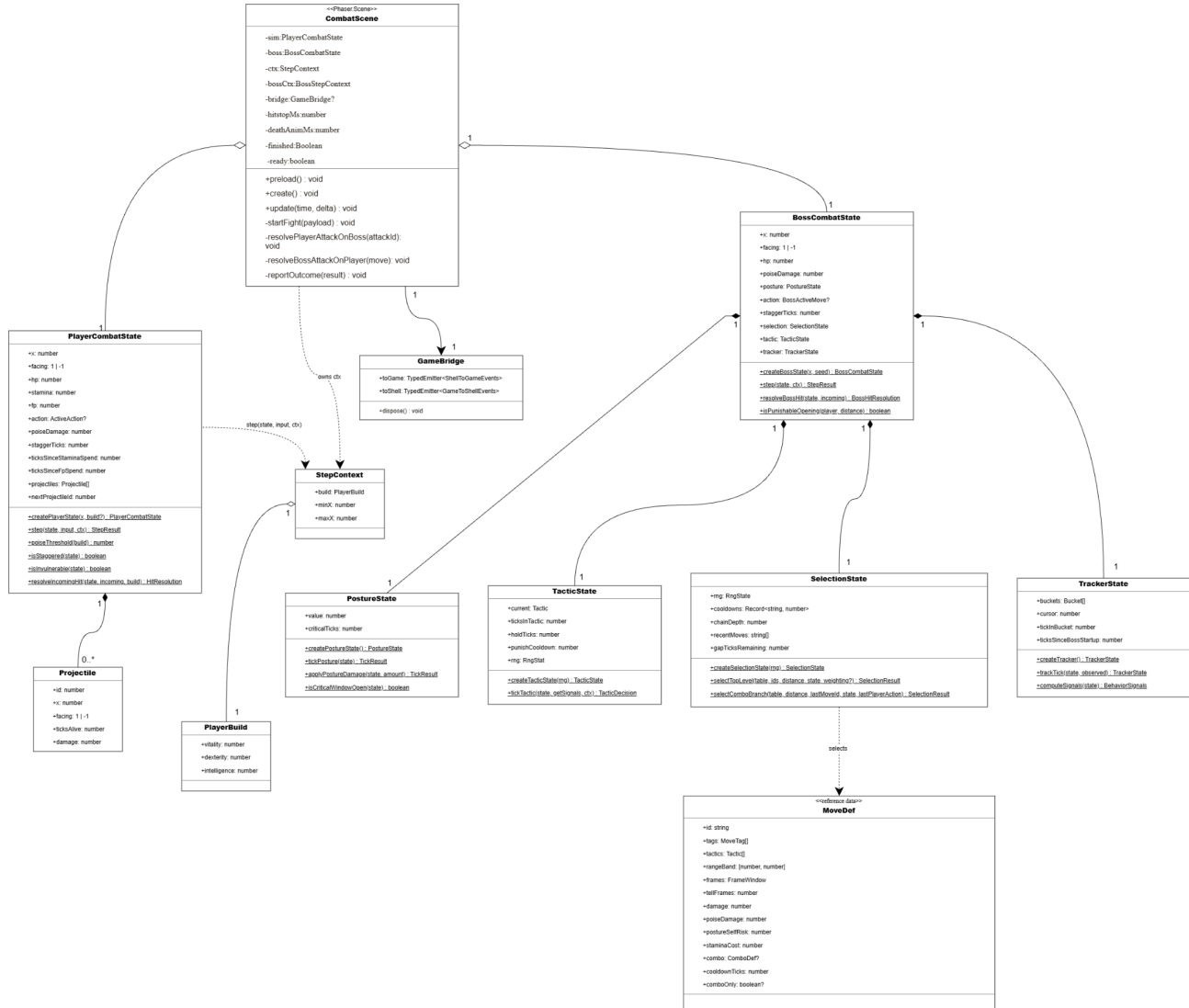
Vitality, Health, Dexterity, and Intelligence: The character attributes that form the primary upgrade system.

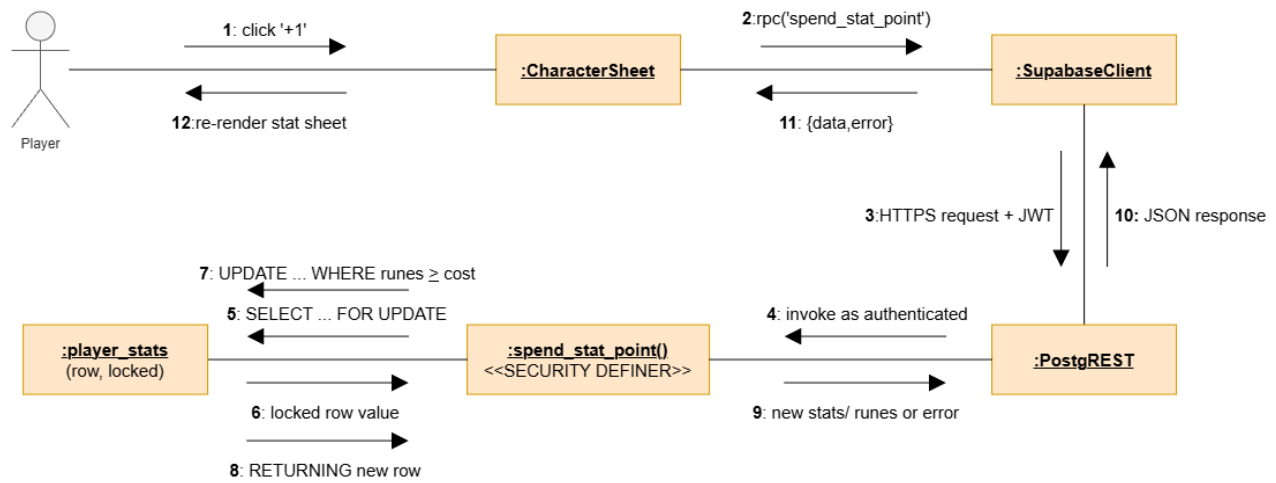
Appendix B: Analysis Models

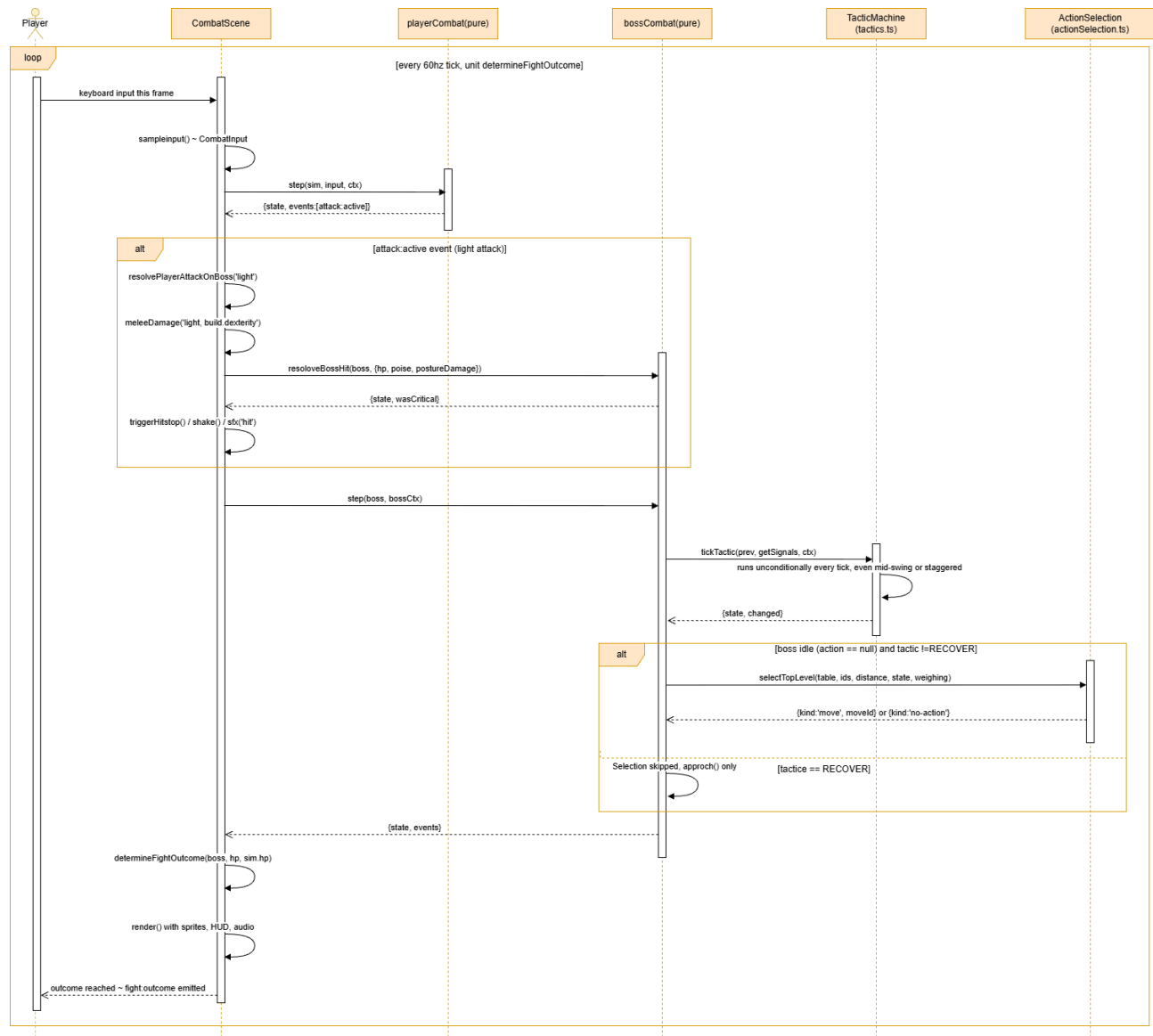
Activity Diagram

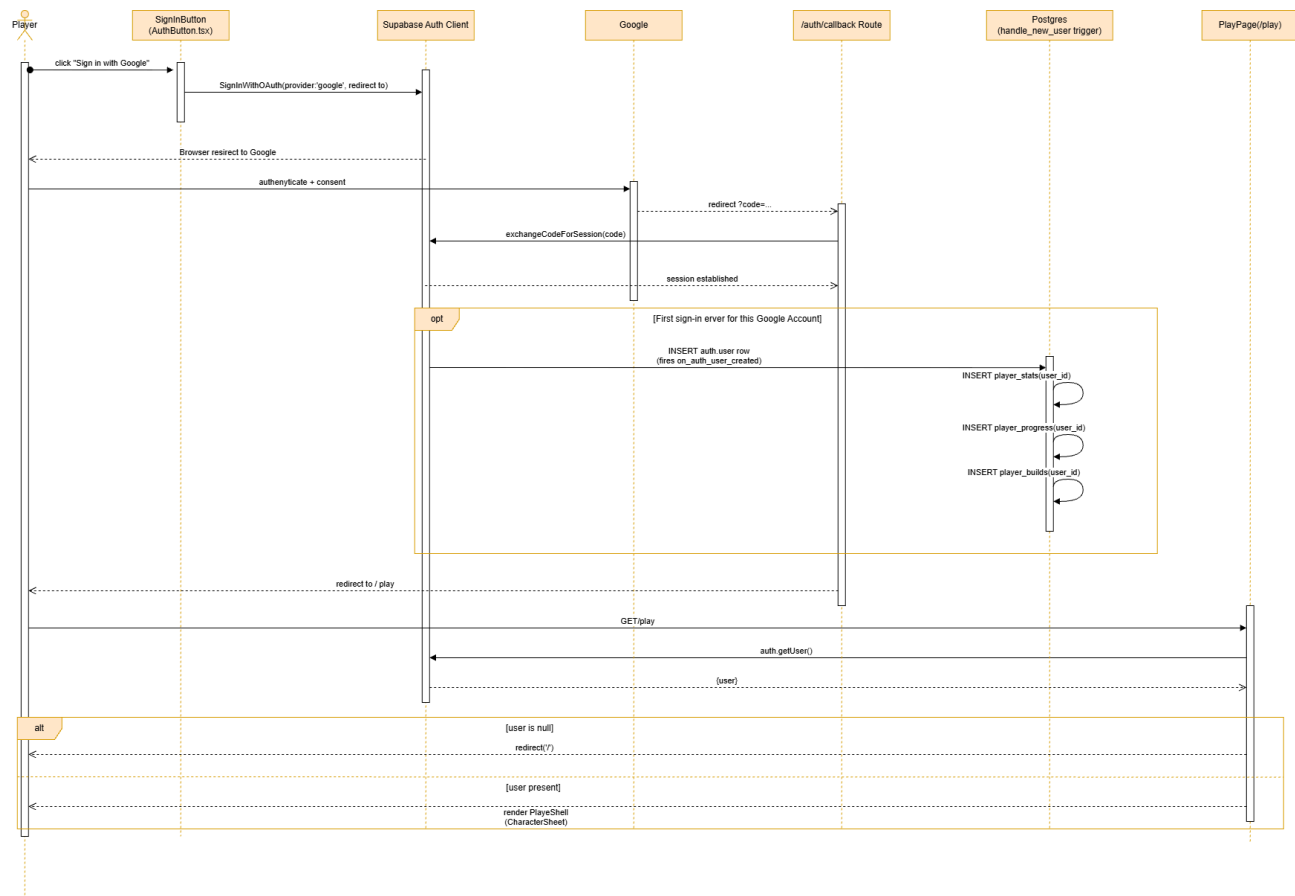


Use Case Diagram

Class Diagram

Communication Diagram

Combat Sequence Diagram

System Sequence Diagram

Appendix C: To Be Determined List

TBD 1 Final balancing values for boss health, damage, stamina costs, cooldowns, and rune costs are to be finalized during implementation and playtesting.

TBD 2 The final list of boss moves and the exact weighting ranges used for per attempt variation are to be finalized during implementation.

TBD 3 The exact wording and presentation format of post death AI explanations are to be finalized during implementation.

TBD 4 Final browser compatibility targets are to be confirmed before release testing.